Git checkout:

1 - the checkout command.

Create a 5 line file where line 3 is a bug.

We do git log –oneline
The final commit is called the HEAD, the HEAD points to the final commit in the commit history.

Cat <file_name> to see the content of the file.

We want to revert all the changes and go back to the second commit.

We grab the second commit ID, and do git checkout <commit_id> → this command updates all the files in the working directory to match the commit ID mentioned.
Issuing this command will put you in a detached head state.

Checking out an old commit is a read-only operation and it's impossible to harm your project, the current state of the project remains untouched in the master branch.

What does detached head mean?
During normal operation, the HEAD points to the last commit on the master, or some other local branch, but when you checkout a commit, the HEAD no longer points to a branch, but to that commit, in our case, a commit ID, this is called a detached head stand.

Read from the detached head message !
Now you can do changes to any files and you can be sure that the current state is unaffected, any commit in the detached head, the commit will not be retained, however, if you want to keep it, you'll need a new branch.

Git checkout master to get back to master

In the detached head, you can compile the project, change files, etc.. without changing anything in the main project, you can confirm that by doing git log --oneline.

- Let's edit the file robots.txt, insert a new line
- Git commit -am "detached commit" (the idea of the commit is that maybe we want to branch out to something new
- View with git status
- If we do git checkout master, this commit will vanish ! nothing we do here will stick unless we do new branch
- Git checkout master → read from the message that we're leaving one commit behind
- In master, git status → show that the commit is gone
- Go back to detached head → cat robots.txt → show that the commit is gone.
- Do the same example again and create a new branch from

2 - Reverting changes:

Git revert HEAD → automatically the editor opens (the default editor right now is the vi editor)

Any revert is a new commit, we can keep it as is or edit the command, save and the commit is done.
Git revert <commit_id> to go back to a specific commit.
Git revert HEAD
Git log online → now you can see the commit reverted, we can confirm that by cat robots.txt (our file has been restored prior to the last commit)
Instead of removing the commit from the history, it will introduce an appended commit and prevent from losing history.

Where can this be useful? If for example you're tracking a bug, and found that it was introduced by a particular commit, instead of manually going in and fixing and committing a new snapshot, you can use the git revert.

Git revert is a safe way to undo changes in a git repository.

3 - git reset → there is no way to get back the original copy, it's one of the only git commands that has the potential to lose your work so it's dangerous and should be used with care.

It's often used to reset changes in the local directory or staging area, however it's never to be used to reset snapshots that were shared with other developers.

Demo:
Create two files, each file is a timetable with 5 entries, each entry is a commit in both files.
Pick up one file and add a line, then:

- git add <file_name>
- git reset <file_name> → show that it's not staged and now in modified, the content of the file is still there then git reset again removes the file from the modified all together.

Git reset without a file name, will reset the changes in all files, for example let's edit the two files and add them to the staging area, again, it will leave the working directory unchanged.

Git reset --hard → unstaged files from the staging area, and the working directory to match the most recent commit.
In addition to un staging changes, the --hard flag tells git to override all changes in the working directory too, in other words, this destroys all uncommitted changes - so make sure you really want to throw away all changes when using --hard option.

So far we looked at the git reset command in the context of the staging area and undoing changes, now we'll look at another use of the git reset command with a commit ID.

Git reset to a specific commit ID:

- Git reset <commit_id>

We can see from the git log that the entire commit was deleted, the files are now in staging area.
What do you guys think the use case here is?
In the 5th commit we changed two files, in a real case scenario there could be 50 changes in 50 different artifact, instead of seeing one commit for all of them, you might want to break them up to say 10 smaller commits with each having 10 changes, to do so:

- Git reset → view the 5th commit that are now in staging area
- Git add first_file

- Git commit -m “commit first file”
- Git commit -m “commit second file”
- (note I didn’t change the content of the files just broke them into different commits).

Another usage is the git reset with hard command.

- Git log → view all 5 commits
- View the content of both files.
- Git reset --hard <commit_id>
- Git log → to view that we’re now at commit #3 (note the message from Git pointing the HEAD)
- View content of files.

Git clean:

Git clean command removes untracked files from your working directory, remember UNTRACKED files, it’s more of a convenient command since it’s easy to see which files are not tracked and remove them manually.
Git clean is often executed along with the git reset with hard option, remember reset only affects tracked files, so a separate command cleans untracked files.
Both commands git reset hard along with git clean allows you to return the working directory to a specific state.

- Mkdir cleanme
- Git init
- Touch file1
- Touch file2
- Touch file3
- Git status
- Git clean -n → dry run, it will show you which files will be deleted without actually deleting them (notice that it’s not mentioning the files that are tracked)
- Git clean -f → will remove untracked files from current directory
    - The -f or force is required unless clean.requreForce configuration option is set to false which by default it’s set to true.
- Note git clean -f will not remove files or folders specified in the .gitignore file.
- Of course you can do git clean -f <path_to_directory> (create a folder and a file) → these are untracked → get out of the directory and execute git clean with path.
- Git clean -df → it will remove both untracked files and untracked directories.
- Git clean -xf → will also remove files in the .gitignore file on top of everything else.

## Git Tag

Lightweight tag:

Git tag v-1.8-rc to create a tag

Git tag → lists a tag

Git log → will show you the tag

When you use git tag it will create a tag from the latest commit

Git show <tag_name> → to show details about the tag

Annotated tag:

Modify a file and commit it

Git tag -a v1.9-rc2 -m “release verison 1.9” (-a = annotated)

If you don’t specify -m git will launch an editor to have you a commit message

How to check what type of tag this is?

First, if it’s an annotated tag, you’ll see information that’s not visible when doing a lightweight tag

The second way is to do
Git cat-file -t <tag_name>
Cat-file provide size or type information about git object
-t specifies that a type tag object is required

If the output is “tag” it means it’s an annotated tag, otherwise the output will be a “commit”

Search a tag:

In a real scenario, we’ll have hundreds of tag

Git tag -l <pattern>
For example git tag -l “v1.*”

Finally you can of course compare tags by doing git diff tag1 tag2

Git Stash:

Git stash → to stash changes (you can add the save flag here)
Git stash list to view the stashed objects

Demo:
Create a few files and commit them to master

Work on master, then stop to work on the hotfix branch → commit the change on hotfix → jump back to master

Start working on another file → stop and need to jump back to hotfix branch → stash again, this time do: git stash save “useful message” → git stash list (notice the reverse order) → go to

hotfix branch and do another commit → jump to master → work on something → stage it → git stash save “msg” → jump to hotfix branch

To identify what’s going on in a stash, do:
Git stash show <paste id>

Add a new file → idea is to show that we can work with untracked files.
The command is git stash -u

Now we’re done with hotfix and we want to go back to master and work on whatever

When we want to retrieve whatever is on the stash: git stash apply
Git stash apply without arguments, will get the latest stash object
To give a specific stash arg we add git stash apply stash@{3} → commit this file.
After we’re done committing the file, we don’t need it in the stash anymore so let’s drop it:
Git stash drop (no args = latest)
Git stash pop = combines apply and drop command, that is gets the stashed files and drops them from the list

Git stash clear = drops all stashes

Git stash branch <branch_name> = will do 4 things:

1) Create a new branch
2) Switch to that branch
3) Does apply from the stash
4) Drops the stash object

Homework:
Merge hotfix branch